# Utility Routines Reference Manual

Модули `utility_routines`, `avproto`, `cli_routines`, `starlet`

Programmer’s Reference Manual

Styled after the VSI OpenVMS RTL (LIB$) Reference

# Table of Contents


### Chapter 1. Utility Routines (UTIL$) Reference

$LOG
__util$deflog
__util$setlogfd
__util$putmsg_fao
__util$log2buf
__util$out
$TRACE
$DUMPHEX
__util$faohex
__util$str2asc
__util$asc2str
__util$cmpasc
__util$getparams
__util$readconfig
__util$showparams
__util$bin2hex
__util$hex2bin
__util$uuid2str
__util$crc32c
$INSQTAIL
$REMQHEAD
__util$movc5
__util$strstr
__util$pattern_match

### Chapter 2. AV Protocol Routines (avproto) Reference

__tlv$hset
__tlv$hget
__tlv$encode_tag
__tlv$decode_tag
__tlv$put
__tlv$get
__tlv$lookup
__tlv$dump

### Chapter 3. Command Language Interface (CLI$) Reference

__cli$parse
__cli$dispatch
__cli$get_value
__cli$present
__cli$cleanup
__cli$show_verbs
__cli$help

### Chapter 4. Time and FAO Services (starlet) Reference

```
__util$fao
__util$faol
__util$bintim
__util$gettim
__util$asctim
__util$add_times
__util$sub_times
```

## Chapter 5. Programming Examples

# Chapter 1. Utility Routines (UTIL$) Reference

This chapter describes the callable routines and macros provided by the `utility_routines` module. Routines follow the OpenVMS condition-value convention: the low bit of a returned status is set on success (`STS$K_SUCCESS`, `STS$K_INFO`) and clear on failure (`STS$K_WARN`, `STS$K_ERROR`, `STS$K_FATAL`). Before including `utility_routines.h`, define `__MODULE__` and, optionally, `__FAC__`.

---

## $LOG

The Write Formatted Log Record macro formats a message under control of a printf-style format string and writes it, with a timestamp, severity and location prefix, to the current log destination.

**Format** `$LOG severity, format [, argument, ...]`

**Returns** `unsigned int` (write only) — The **severity** value passed in, so that the macro may be used directly in a `return` statement.

| Argument | Type | Access | Description |
|---|---|---|---|
| `severity` | `unsigned int` | read only | Severity of the record: one of `STS$K_SUCCESS`, `STS$K_INFO`, `STS$K_WARN`, `STS$K_ERROR` or `STS$K_FATAL`. The corresponding letter (S/I/W/E/F) is emitted in the record prefix. |
| `format` | `const char *` | read only | A printf-style, zero-terminated format string. In addition to the standard conversions the special conversion `%C` emits the one-letter severity code. |
| `argument` | `...` | read only | Optional arguments consumed by the conversions in **format**. |

$LOG is the normal entry point for diagnostics. It expands to a call to `__util$logd()`, automatically supplying the facility (`__FAC__`), module (`__MODULE__`), function and line number of the call site. Each record is prefixed with `DD-MM-YYYY HH:MM:SS.msec <pid/tid> [module\function:line] %FAC-S:` and written to the destination selected by `__util$deflog()` (standard output by default).

The caller must define `__MODULE__` (and optionally `__FAC__`) before including `utility_routines.h`.

**Condition values**

`STS$K_*` The routine returns the **severity** argument unchanged.

## __util$deflog

The Define Log Destination routine redirects subsequent log output to a file and, optionally, to a remote SYSLOG host.

**Format** `int __util$deflog (logfile, sloghost)`

**Returns** `int` (write only) — A status code; success has the low bit set.

| Argument | Type | Access | Description |
|---|---|---|---|
| `logfile` | `const char *` | read only | Zero-terminated path of the log file to append to, or `NULL` to keep standard output. The file is opened with `O_CLOEXEC`. |
| `sloghost` | `const char *` | read only | Zero-terminated IPv4 address of a SYSLOG server, or `NULL` for none. |

Called once during start-up, before the first `$LOG`. When a SYSLOG host is given, records are additionally forwarded as UDP datagrams to the standard syslog port.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | Routine successfully completed. |
| `STS$K_ERROR` | The log file could not be opened; `errno` indicates the reason. |

## __util$setlogfd

The Set Log Descriptor routine points the logging at an already-open descriptor, without opening a file or seizing the standard streams.

**Format** `int __util$setlogfd (a_fd)`

**Returns** `int` (write only) — A status code; success has the low bit set.

| Argument | Type | Access | Description |
|---|---|---|---|
| `a_fd` | `int` | read only | An already-open descriptor to receive the log records. A negative value is rejected. |

The companion of `__util$deflog()`, which takes a file name and seizes both standard output and standard error with `dup2()` - right for a daemon, wrong for a utility whose standard output carries the data it was run to produce. Such a utility calls `__util$setlogfd(STDERR_FILENO)` once, so its `$LOG` / `$TRACE` records go to the standard error while its standard output stays clean.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | Routine successfully completed. |
| `STS$K_ERROR` | The descriptor was negative; the current sink is left unchanged. |

## __util$putmsg_fao

The Put Message (FAO) routine signals a message from the registered catalogue, taking its parameters through the FAO formatting service rather than a printf format.

**Format** `unsigned __util$putmsg_fao (sts, prmcnt, ...)`

**Returns** `unsigned` (write only) — The argument, returned unchanged so the routine can be the operand of a return statement.

| Argument | Type | Access | Description |
|---|---|---|---|
| `sts` | `unsigned` | read only | The condition value to be signalled. Its facility and severity frame the record; its message number selects the text from the catalogue. |
| `prmcnt` | `int` | read only | The number of FAO parameters that follow, 0 to 16. Written for the caller by the `$PUTMSG_FAO()` macro - see below. |
| `...` | `fao_prm_t` | read only | The FAO parameters, each a 64-bit `fao_prm_t`, substituted into the record's FAO control string (`!AS`, `!UL`, `!XL`, `!UQ` and so on). |

Signals a catalogue message as `__util$putmsg()` does, but the record's text is an FAO control string rendered by `__util$faol()`, so a message may carry typed directives - `!UL`, `!XL`, `!AS`, `!UQ` - in place of a printf format. The catalogue is the one registered by `__util$inimsg()`; a value with no record yields a default diagnostic. The record is matched by its facility and message number, the severity bits ignored, so one status may be signalled `INFO` in one place and `WARNING` in another from a single record. A found record is framed in the OpenVMS form `%<facility>-<S>-<text>`: the facility name from the descriptor it was registered with (or `NONAME`), and the severity letter from `$SEV` of the status. The status is returned too, so its severity drives the caller's own control flow. The record is framed with the timestamp, the process id and, in a debug build, the module and line, and written to the destination chosen by `__util$deflog()` / `__util$setlogfd()`.

Parameters travel by varargs and are read back as 64-bit `fao_prm_t`, so each must be cast to that type - an un-cast 32-bit value is undefined behaviour. The `$PUTMSG_FAO()` macro applies the cast for the caller and derives the count from the list, so a value is written plain: `$PUTMSG_FAO(sts, l_seq, l_disp)`. A count written by hand can disagree with the list; a count derived from it cannot.

**Condition values**

| | |
|---|---|
| `(returns sts)` | The routine returns its argument unchanged. |

## __util$log2buf

The Format Log Record into Buffer routine formats a log record exactly as $LOG would, but delivers it into a caller-supplied buffer instead of the log destination.

**Format** `unsigned __util$log2buf (out, outsz, outlen, fac, severity, format [, argument, ...])`

**Returns** `unsigned int` (write only) — The **severity** value passed in.

| Argument | Type | Access | Description |
|---|---|---|---|
| `out` | `void *` | write only | Address of the output buffer. |
| `outsz` | `int` | read only | Size of **out** in bytes. |
| `outlen` | `int *` | write only | Receives the number of bytes written to **out**. |
| `fac` | `const char *` | read only | Facility name for the prefix. |
| `severity` | `unsigned int` | read only | Record severity. |
| `format` | `const char *` | read only | printf-style format string. |
| `argument` | `...` | read only | Optional format arguments. |

Useful when the formatted text is needed as a string (for a protocol field, a UI element, and so on) rather than written to the log.

**Condition values**

`STS$K_*` — The routine returns the **severity** argument.

## __util$out

The Write Plain Record routine writes a printf-formatted line to the log destination without the timestamp/severity prefix.

**Format** `unsigned __util$out (format [, argument, ...])`

**Returns** `unsigned int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| `format` | `char *` | read only | printf-style format string. |
| `argument` | `...` | read only | Optional format arguments. |

Intended for banners, tables and other output that should not carry the diagnostic prefix.

**Condition values**

`STS$K_SUCCESS` — Routine successfully completed.

## $TRACE

The Write Trace Record macro emits a diagnostic trace record in debug builds and compiles to nothing when NDEBUG is defined.

**Format** `$TRACE format [, argument, ...]`

**Returns** None.

| Argument | Type | Access | Description |
|---|---|---|---|
| `format` | `const char *` | read only | printf-style format string. |
| `argument` | `...` | read only | Optional format arguments. |

$TRACE expands to a call to `__util$trace()` carrying the call-site module, function and line. When the translation unit is compiled with `-DNDEBUG` the macro becomes a no-op, so trace points may be left in production code at no cost. The related `$IFTRACE(cond, ...)` emits the record only when **cond** is true.

## $DUMPHEX

The Dump Buffer in Hexadecimal macro writes a formatted hex/ASCII dump of a memory region to the log destination.

**Format** `$DUMPHEX buffer, length`

**Returns** None.

| Argument | Type | Access | Description |
|---|---|---|---|
| `buffer` | `const void *` | read only | Address of the region to dump. |
| `length` | `unsigned short` | read only | Number of bytes to dump. |

Expands to `__util$dumphex()`, supplying the call-site function and line. Output is laid out sixteen octets per line as `offset: hex ... ascii`.

## __util$faohex

The Format One Hexadecimal Dump Line routine formats up to sixteen octets as a single hex-and-ASCII dump line.

**Format** `int __util$faohex (src, srclen, out, outsz, outlen)`

**Returns** `int` (write only) — The number of bytes written, or 0 on error.

| Argument | Type | Access | Description |
|---|---|---|---|
| `src` | `const void *` | read only | Address of the octets to format. |
| `srclen` | `unsigned short` | read only | Number of octets; values above 16 are clamped to 16. |
| `out` | `char *` | write only | Address of the output buffer. |
| `outsz` | `unsigned short` | read only | Size of **out**; must be at least 67 bytes or the routine returns 0. |
| `outlen` | `unsigned short *` | write only | Receives the number of bytes written to **out**. |

Produces one line of the form `XX XX ... XX ....ascii.....`. Unused columns are space-padded and non-printable octets are shown as a dot.

**Condition values**

| | |
|---|---|
| `>= 0` | Number of bytes written; 0 indicates **outsz** was too small. |

## __util$str2asc

The Copy String to Counted String routine copies a zero-terminated string into an ASCIC (counted) string container.

**Format** `int __util$str2asc (src, dst)`

**Returns** `int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| `src` | `const char *` | read only | Zero-terminated source string. |
| `dst` | `ASC *` | write only | Destination ASCIC container. |

The length is taken from the source up to `ASC$K_SZ` (255) octets. The counterpart `__util$asc2str()` performs the reverse conversion.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | Routine successfully completed. |

## __util$asc2str

The Copy Counted String to String routine copies an ASCIC container into a zero-terminated string.

**Format** `int __util$asc2str (src, dst)`

**Returns** `int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| `src` | `ASC *` | read only | Source ASCIC container. |
| `dst` | `char *` | write only | Destination buffer; must hold `src->len + 1` bytes. |

A terminating NUL is appended.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | Routine successfully completed. |

## __util$cmpasc

The Compare Counted Strings routine compares two ASCIC containers for equality.

**Format** `int __util$cmpasc (a, b)`

**Returns** `int` (write only) — Non-zero if the strings are equal.

| Argument | Type | Access | Description |
|---|---|---|---|
| `a` | `ASC *` | read only | First ASCIC string. |
| `b` | `ASC *` | read only | Second ASCIC string. |

Compares lengths first, then the bytes. The related `__util$cmpasc_blind()` performs a case-insensitive comparison.

## __util$getparams

The Parse Command-Line Options routine fills a table of typed option descriptors from the program argument vector.

**Format** `int __util$getparams (argc, argv, opts)`

**Returns** `int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| `argc` | `int` | read only | Argument count, as passed to main(). |
| `argv` | `char *[]` | read only | Argument vector. |
| `opts` | `const OPTS *` | modify | Address of an array of `OPTS` descriptors terminated by `OPTS_NULL`. Each descriptor names an option, points at the receiving variable, its size and its `OPTS$K_*` type. |

Accepts options written as `-name=value`, `/name=value` or `--name=value`. Option names may be abbreviated to any unambiguous prefix; an argument longer than the option name does not match it. Values are converted according to the descriptor type (`OPTS$K_INT`, `OPTS$K_STR`, `OPTS$K_OPT`, `OPTS$K_PORT`, `OPTS$K_IP4`, `OPTS$K_CONF`, `OPTS$K_PWD`). A descriptor of type `OPTS$K_CONF` causes the named configuration file to be read immediately through `__util$readconfig()`.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | All recognised options were parsed. |
| `STS$K_ERROR` | An option value could not be converted, or a mandatory option was missing. |

## __util$readconfig

The Read Configuration File routine fills a table of typed option descriptors from a text configuration file.

**Format** `int __util$readconfig (fconf, opts)`

**Returns** `int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| `fconf` | `const char *` | read only | Zero-terminated path of the configuration file. An empty or NULL path is an error. |
| `opts` | `OPTS *` | modify | Address of an `OPTS` table terminated by `OPTS_NULL`, as for `__util$getparams()`. |

The file contains one option per line in the same `-name=value` form accepted on the command line; text following a `!` is treated as a comment. The same option table may therefore be shared between the command line and the configuration file.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | The file was read and parsed. |
| `STS$K_ERROR` | The file name was empty, the file could not be opened, or a value was invalid. |

## __util$showparams

The Display Option Values routine writes the current value of every descriptor in an option table to the log.

**Format** `int __util$showparams (opts)`

**Returns** `int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| `opts` | `const OPTS *` | read only | Address of an `OPTS` table terminated by `OPTS_NULL`. |

Password descriptors (`OPTS$K_PWD`) are shown masked.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | Routine successfully completed. |

## __util$bin2hex

The Convert Binary to Hexadecimal routine converts a sequence of octets to a lowercase hexadecimal string.

**Format** `int __util$bin2hex (src, dst, srclen)`

**Returns** `int` (write only) — The length of the hexadecimal string, `srclen * 2`.

| Argument | Type | Access | Description |
|---|---|---|---|
| `src` | `const void *` | read only | Address of the source octets. |
| `dst` | `void *` | write only | Destination buffer; must hold `srclen * 2 + 1` bytes. A terminating NUL is written. |
| `srclen` | `unsigned short` | read only | Number of source octets. |

The convenience macro `$BIN2HEX(s, d, l)` supplies the casts. `__util$hex2bin()` performs the reverse conversion.

**Condition values**

| | |
|---|---|
| `>= 0` | The number of hexadecimal characters written. |

## __util$hex2bin

The Convert Hexadecimal to Binary routine converts a hexadecimal string back to octets.

**Format** `int __util$hex2bin (src, dst, srchexlen)`

**Returns** `int` (write only) — The number of octets written.

| Argument | Type | Access | Description |
|---|---|---|---|
| `src` | `const void *` | read only | Address of the hex string. |
| `dst` | `void *` | write only | Destination octet buffer. |
| `srchexlen` | `unsigned short` | read only | Number of hexadecimal characters to convert (two per output octet). |

Both upper- and lowercase hexadecimal digits are accepted.

**Condition values**

| | |
|---|---|
| `>= 0` | The number of octets written. |

## __util$uuid2str

The Convert UUID to String routine formats a 16-octet UUID as its canonical text representation.

**Format** `int __util$uuid2str (uuid, buf, bufsz)`

**Returns** `int` (write only) — The length of the formatted string.

| Argument | Type | Access | Description |
|---|---|---|---|
| `uuid` | `void *` | read only | Address of the 16-octet UUID. |
| `buf` | `char *` | write only | Destination buffer; 37 bytes are sufficient. |
| `bufsz` | `int` | read only | Size of **buf** in bytes. |

The result has the form `xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx`.

**Condition values**

| | |
|---|---|
| `>= 0` | The number of characters written. |

## __util$crc32c

The Compute CRC-32 Checksum routine computes a 32-bit cyclic redundancy check over a memory region.

**Format** `unsigned __util$crc32c (crc, buf, buflen)`

**Returns** `unsigned int` (write only) — The 32-bit checksum.

| Argument | Type | Access | Description |
|---|---|---|---|
| `crc` | `unsigned int` | read only | Initial value. Pass 0 to begin a new checksum; pass the result of a previous call to continue an incremental computation. |
| `buf` | `const void *` | read only | Address of the data. |
| `buflen` | `size_t` | read only | Number of octets. |

Despite its name, this routine computes the standard IEEE 802.3 / zlib CRC-32 (reflected polynomial 0xEDB88320), **not** Castagnoli CRC-32C; the check value `__util$crc32c(0, "123456789", 9)` is `0xCBF43926`. On ARMv8 targets that advertise the CRC extension the hardware `crc32` instructions are used; elsewhere a portable slice-by-eight implementation is used. The result is identical on all platforms and byte orders.

**Condition values**

| | |
|---|---|
| `longword` | The checksum value (this routine does not use the condition-value convention). |

## $INSQTAIL

The Insert Entry at Tail of Queue macro links an entry onto the tail of a doubly-linked queue under spinlock protection.

**Format** `$INSQTAIL queue, entry, count`

**Returns** `int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| `queue` | `__QUEUE *` | modify | Address of the queue header. |
| `entry` | `void *` | modify | Address of the entry, whose first member must be an `ENTRY`. The entry must be zeroed before its first insertion. |
| `count` | `unsigned *` | write only | Receives the number of elements in the queue before the insertion. |

Expands to `__util$insqtail()`. `$INSQHEAD` inserts at the head instead. The entry structure must begin with an `ENTRY` member; the queue header is a `__QUEUE` initialised with `QUEUE_INITIALIZER`.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | The entry was inserted. |
| `UTIL$S_INQUE` | The entry is already linked into another queue. |
| `UTIL$S_INVARG` | A NULL argument was supplied. |

## $REMQHEAD

The Remove Entry from Head of Queue macro unlinks the entry at the head of a doubly-linked queue.

**Format** `$REMQHEAD queue, entry, count`

**Returns** `int` (write only) — A status code (see below).

| Argument | Type | Access | Description |
|---|---|---|---|
| `queue` | `__QUEUE *` | modify | Address of the queue header. |
| `entry` | `void **` | write only | Receives the address of the removed entry. It is **not** written when the queue is empty. |
| `count` | `unsigned *` | write only | Receives the number of elements in the queue before the removal. |

Expands to `__util$remqhead(); $REMQTAIL` removes from the tail. The routine returns `STS$K_SUCCESS` even when the queue is empty, in which case **entry** is left unchanged and **count** is zero. A dequeue loop must therefore test **count** to detect the empty condition, otherwise it will not terminate.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | The operation completed. An entry was removed only if **count** was non-zero. |
| `STS$K_ERROR` | A NULL argument was supplied or the lock could not be taken. |

## __util$movc5

The Move Characters with Fill routine copies a source region to a destination region, padding with a fill octet or truncating as required. It mirrors the OpenVMS LIB$MOVC5 primitive.

**Format** `int __util$movc5 (srclen, source, fill, dstlen, dest)`

**Returns** `int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| `srclen` | `unsigned short *` | read only | Address of the source length in octets. |
| `source` | `char *` | read only | Address of the source region. |
| `fill` | `unsigned char *` | read only | Address of the fill octet, used when the destination is longer than the source. |
| `dstlen` | `unsigned short *` | read only | Address of the destination length in octets. |
| `dest` | `char *` | write only | Address of the destination region. |

If the destination is shorter than the source, the highest-addressed source octets are not moved. If it is longer, the remainder is set to the fill octet.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | Routine successfully completed. |

## __util$strstr

The Locate Substring by Length routine searches a counted region for the first occurrence of a counted pattern, without relying on NUL termination.

**Format** `char * __util$strstr (s1, s1len, s2, s2len)`

**Returns** `char *` (write only) — Address of the first occurrence, or `NULL` if the pattern was not found.

| Argument | Type | Access | Description |
|---|---|---|---|
| `s1` | `char *` | read only | Address of the region to search. |
| `s1len` | `size_t` | read only | Length of the region. |
| `s2` | `char *` | read only | Address of the pattern. |
| `s2len` | `size_t` | read only | Length of the pattern. |

Both operands may contain embedded NUL octets.

**Condition values**

| | |
|---|---|
| `address` | A pointer into **s1**, or `NULL`. |

## __util$pattern_match

The Match String Against Pattern routine tests whether a string matches a wildcard pattern.

**Format** `int __util$pattern_match (str, pattern)`

**Returns** `int` (write only) — Non-zero if the string matches.

| Argument | Type | Access | Description |
|---|---|---|---|
| `str` | `char *` | read only | Zero-terminated subject string. |
| `pattern` | `char *` | read only | Zero-terminated pattern. `*` matches any run of characters and `%` matches exactly one character. |

Comparison is case-sensitive.

**Condition values**

| | |
|---|---|
| `1` | The string matches the pattern. |
| `0` | The string does not match. |

# Chapter 2. AV Protocol Routines (avproto) Reference

This chapter describes the routines of the `avproto` module, a compact binary protocol built from a fixed 20-octet header followed by a variable number of Tag-Length-Value (TLV) records. All multi-octet fields are held in network byte order. A Protocol Data Unit is addressed through the `AVPROTO_PDU` type; routines use the `__tlv$` prefix. Tag types are the `TAG$K_*` constants (`BBLOCK`=0, `WORD`=3, `LONGWORD`=4, `QWORD`=5, `UUID`=8).

---

## __tlv$hset

The Set PDU Header routine initialises the fixed header of a Protocol Data Unit and resets its payload length to zero.

**Format** `int __tlv$hset (pdu, sig, u_csr, u_seq)`

**Returns** `int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| `pdu` | `void *` | write only | Address of the PDU buffer. |
| `sig` | `const unsigned char *` | read only | Eight-octet protocol signature written into the header. |
| `u_csr` | `unsigned int` | read only | Command/status register value. |
| `u_seq` | `unsigned int` | read only | PDU sequence number. |

Must be called before the first `__tlv$put()`, because it clears the payload length. All header fields are stored in network byte order.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | Routine successfully completed. |

---

## __tlv$hget

The Get PDU Header routine validates a PDU signature and extracts the header fields.

**Format** `int __tlv$hget (pdu, sig, u_len, u_csr, u_seq)`

**Returns** `int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| `pdu` | `void *` | read only | Address of the received PDU. |
| `sig` | `const unsigned char *` | read only | Expected eight-octet signature; a mismatch is an error. |
| `u_len` | `unsigned int *` | write only | Receives the payload length declared in the header. |
| `u_csr` | `unsigned int *` | write only | Receives the command/status register. |
| `u_seq` | `unsigned int *` | write only | Receives the sequence number. |

All returned fields are converted from network to host byte order.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | The signature matched and the fields were extracted. |
| `STS$K_ERROR` | The signature did not match. |

## __tlv$encode_tag

The Encode TLV Tag routine packs a tag type and tag identifier into the sixteen-bit tag field of a TLV.

**Format** `unsigned short __tlv$encode_tag (v_type, v_tag)`

**Returns** `unsigned short` (write only) — The packed `w_tag` field, in network byte order.

| Argument | Type | Access | Description |
|---|---|---|---|
| `v_type` | `unsigned short` | read only | Tag type; one of the `TAG$K_*` constants (four bits). |
| `v_tag` | `unsigned short` | read only | Application-specific tag identifier (twelve bits). |

The layout is `[ type : 4 | id : 12 ]`, returned in network byte order.

## __tlv$decode_tag

The Decode TLV Tag routine extracts the tag type and tag identifier from a TLV tag field.

**Format** `unsigned short __tlv$decode_tag (w_tag, v_type, v_tag)`

**Returns** `unsigned short` (write only) — The tag field in host byte order.

| Argument | Type | Access | Description |
|---|---|---|---|
| `w_tag` | `unsigned short` | read only | The `w_tag` field, in network byte order. |
| `v_type` | `unsigned short *` | write only | Receives the tag type. |
| `v_tag` | `unsigned short *` | write only | Receives the tag identifier. |

The high four bits give the type and the low twelve bits the identifier.

## __tlv$put

The Append TLV routine appends one Tag-Length-Value record to a PDU.

**Format** `int __tlv$put (pdu, pdusz, v_tag, v_type, val, valsz)`

**Returns** `int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| `pdu` | `void *` | modify | Address of the PDU buffer. |
| `pdusz` | `unsigned short` | read only | Total size of the PDU buffer in octets. |
| `v_tag` | `unsigned int` | read only | Application tag identifier. |
| `v_type` | `unsigned int` | read only | Tag type: `TAG$K_BBLOCK`, `TAG$K_WORD`, `TAG$K_LONGWORD`, `TAG$K_QWORD` or `TAG$K_UUID`. |
| `val` | `void *` | read only | Address of the value to store. |
| `valsz` | `unsigned int` | read only | Size of the value in octets. It may be zero for the fixed-width integer types. |

Word, longword and quadword values are stored in network byte order. The routine appends after the current payload and updates the header length. It fails if the value does not fit in the remaining space of the buffer.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | The record was appended. |
| `STS$K_ERROR` | The buffer is too small, or the header declares a corrupt length. |

## __tlv$get

The Extract TLV Value routine finds a TLV by tag and copies its value into a caller buffer.

**Format** `int __tlv$get (pdu, pdusz, context, v_tag, v_type, val, valsz)`

**Returns** `int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| `pdu` | `void *` | read only | Address of the received PDU. |
| `pdusz` | `unsigned int` | read only | The **real** size of the PDU area, that is the number of octets actually received. Every TLV is validated against this bound, so a malformed or hostile PDU cannot cause an out-of-bounds access. Do not pass the header length here; it is not trusted. |
| `context` | `int *` | modify | Iteration context. Set to -1 for the first call; if reused across calls the search advances forward through the PDU. |
| `v_tag` | `unsigned int` | read only | Tag identifier to find. |
| `v_type` | `unsigned int *` | write only | Receives the tag type of the located record. |
| `val` | `void *` | write only | Address of the buffer to receive the value. |
| `valsz` | `unsigned int *` | modify | On entry, the size of **val**; on return, the number of octets copied. For variable-length types (BBLOCK, UUID) this argument is mandatory; omitting it is an error. |

Word, longword and quadword values are converted from network to host byte order. Use `__tlv$lookup()` instead when a pointer to the value is wanted without a copy.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | The record was found and its value copied. |
| `STS$K_WARN` | No record with the given tag was found. |
| `STS$K_ERROR` | The destination is too small, or **valsz** was omitted for a variable-length type. |

## __tlv$lookup

The Locate TLV Value routine finds a TLV by tag and returns a pointer to its value inside the PDU, without copying.

**Format** `int __tlv$lookup (pdu, pdusz, context, v_tag, v_type, val, valsz)`

**Returns** `int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| `pdu` | `void *` | read only | Address of the received PDU. |
| `pdusz` | `unsigned int` | read only | Real size of the PDU area, as for `__tlv$get()`. |
| `context` | `int *` | modify | Iteration context; -1 on the first call. |
| `v_tag` | `unsigned int` | read only | Tag identifier to find. |
| `v_type` | `unsigned int *` | write only | Receives the tag type. |
| `val` | `void **` | write only | Receives a pointer to the value inside the PDU. The data is **not** byte-order converted. |
| `valsz` | `unsigned int *` | write only | Receives the length of the value. |

The returned pointer references the caller's PDU buffer and remains valid only as long as that buffer does. Suitable for large octet blocks that need not be copied.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | The record was found. |
| `STS$K_WARN` | No record with the given tag was found. |

## __tlv$dump

The Dump PDU routine writes a human-readable listing of every TLV in a PDU to the log.

**Format** `void __tlv$dump (pdu, pdusz)`

**Returns** None.

| Argument | Type | Access | Description |
|---|---|---|---|
| `pdu` | `void *` | read only | Address of the PDU. |
| `pdusz` | `unsigned int` | read only | Real size of the PDU area, as for `__tlv$get()`. |

Each record is listed with its tag, type, length and a hexadecimal dump of the value. Intended for debugging.

# Chapter 3. Command Language Interface (CLI$) Reference

This chapter describes the callable routines of the `cli_routines` module, a DCL-style command line parser. A command has the grammar `verb [p1 p2 ... p8] [/qualifier[=value] ...]`. Routines use the `__cli$` prefix; the `CLI$K_*` constants keep the `CLI$` prefix. Verbs are described by a table of `CLI_VERB` records; each parameter and qualifier is a `CLI_PQDESC` carrying a value type. Every value is validated against its type:

| | |
|---|---|
| `CLI$K_FILE = 1` | A file specification (any string). |
| `CLI$K_DATE = 2` | A date/time string. |
| `CLI$K_NUM = 3` | A number (octal, decimal or hex). |
| `CLI$K_IPV4 = 4` | An IPv4 address. |
| `CLI$K_IPV6 = 5` | An IPv6 address. |
| `CLI$K_OPT = 7` | A flag (no value). |
| `CLI$K_QSTRING = 8` | A quoted string. |
| `CLI$K_UUID = 9` | A canonical UUID. |
| `CLI$K_DEVICE = 0xA` | A device under `/dev/`. |
| `CLI$K_KWD = 0xB` | One of a list of keywords. |
| `CLI$K_TIME = 0xC` | An absolute or delta time, validated via starlet `__util$bintim`. |
| `CLI$K_MAC = 0xD` | A MAC address: `aa:bb:cc:dd:ee:ff` or `aa-bb-...` |

Processing options are `CLI$M_OPTRACE` (trace the parse) and `CLI$M_OPSIGNAL` (emit a diagnostic for each error). A qualifier descriptor may carry `CLI$M_NEGATABLE` (accepts `/NO<name>`), `CLI$M_LIST` (a value list `/Q=(a,b,c)`) or `CLI$M_REQUIRED` (must be supplied); a non-empty `defval` supplies a default. A built-in `/HELP` (also `-h`/`--help`) prints usage via `__cli$help`. Routines follow the condition-value convention (low bit set on success).

## __cli$parse

The Parse Command Line routine matches the program arguments against a command definition table, validating verbs, positional parameters and qualifiers, and builds a context that records the result.

**Format** `int __cli$parse (verbs, opts, argc, argv, clictx)`

**Returns** `int` (write only) — A status code; success has the low bit set.

| Argument | Type | Access | Description |
|---|---|---|---|
| `verbs` | `CLI_VERB *` | read only | Address of the command definition table: an array of `CLI_VERB` descriptors, each naming a verb and pointing at its parameter and qualifier tables, an action routine and an optional chain of sub-verbs. |
| `opts` | `int` | read only | Processing options: `CLI$M_OPTRACE` traces the parse, `CLI$M_OPSIGNAL` emits a diagnostic for every syntax error. |
| `argc` | `int` | read only | Argument count. |
| `argv` | `char **` | read only | Argument vector. |
| `clictx` | `void **` | write only | Receives the address of a newly allocated parse context. On failure it is released and set to `NULL`. |

Accepts the grammar `verb [p1 p2 ... p8] [/qualifier[=value] ...]`. Verb and qualifier names may be abbreviated to any unambiguous prefix. Every value is checked against the declared type of its parameter or qualifier (see `__cli$val_type` types below). The resulting context is consumed by `__cli$dispatch()` and `__cli$get_value()` and must be released with `__cli$cleanup()`.

A verb may nest: it can carry a chain of sub-verbs, so `SHOW STATISTICS` resolves `STATISTICS` in `SHOW`'s sub-verb list. A sub-verb follows its verb immediately - no qualifier may stand between them.

A qualifier is written `/name` (VMS) or `--name` (GNU long form); a single `-` is not a qualifier prefix, so a value may begin with one. Parameters are positional and are given before the qualifiers. A token that names a defined qualifier stops the positional scan and is taken as that qualifier, so an optional parameter may be omitted in its favour - `/etc/passwd`, being longer than any qualifier name, is not one and is kept as the parameter's value, while `/csv` is the qualifier. A bare token that fills neither a parameter nor a qualifier is reported (`Unrecognized token`) rather than dropped in silence; declare a parameter `CLI$M_REQUIRED` where its absence must itself be an error.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | The command line parsed cleanly. |
| `STS$K_ERROR` | A value failed type validation, or memory could not be allocated. |
| `STS$K_FATAL` | An unrecognised or ambiguous verb/qualifier, or too few arguments. |

## __cli$dispatch

The Dispatch Command routine invokes the action routine associated with the parsed verb.

**Format** `int __cli$dispatch (clictx)`

**Returns** `int` (write only) — The status returned by the action routine.

| Argument | Type | Access | Description |
|---|---|---|---|
| clictx | CLI_CTX * | read only | Address of a context produced by __cli$parse(). |

The action routine registered in the matched `CLI_VERB` is called with the context and the verb's `act_arg`. Within the action routine, parameter and qualifier values are retrieved with `__cli$get_value()`.

**Condition values**

| | |
|---|---|
| STS$K_SUCCESS | The action routine completed successfully. |
| (other) | Whatever condition value the action routine returns. |

## __cli$get_value

The Get Parameter or Qualifier Value routine retrieves the value string bound to a given parameter or qualifier descriptor.

**Format** `int __cli$get_value (clictx, pq, val)`

**Returns** `int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| clictx | CLI_CTX * | read only | Address of the parse context. |
| pq | CLI_PQDESC * | read only | Address of the parameter or qualifier descriptor whose value is wanted. |
| val | ASC * | write only | Receives the value as an ASCIC string. |

Accepts the grammar and is consumed after `__cli$parse()`. For a `CLI$M_LIST` qualifier (e.g. `/Q=(a,b,c)`) successive calls return successive values and then report absence. When the item is not present and its descriptor carries a non-empty `defval`, that default value is returned instead.

**Condition values**

| | |
|---|---|
| STS$K_SUCCESS | A value was returned. |
| STS$K_WARN | The parameter or qualifier was not present on the command line. |

## __cli$present

The Test Presence routine reports whether a parameter or qualifier was supplied on the command line, and whether it was given in its negated form.

**Format** `int __cli$present (CLI_CTX *clictx, CLI_PQDESC *pq, unsigned *state)`

**Returns** `int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| `clictx` | `CLI_CTX *` | read only | Address of the parse context. |
| `pq` | `CLI_PQDESC *` | read only | The parameter or qualifier descriptor to test. |
| `state` | `unsigned *` | write only | Optional; receives the item state - `CLI$M_PRESENT`, plus `CLI$M_NEGATED` when the qualifier was given as `/NO<name>`. Set to zero when the item is absent. When a qualifier is repeated - most usefully `/HIGHLIGHT` then `/NOHIGHLIGHT` - the last occurrence on the command line wins, as it does in the DCL. |

The analogue of the OpenVMS CLI$PRESENT service. It is the companion of `__cli$get_value()` for a negatable qualifier (`CLI$M_NEGATABLE`), where the affirmative and negated forms are told apart by `CLI$M_NEGATED` in **state** rather than by a value.

When a qualifier is given more than once, the state of the **last** occurrence is reported, so a trailing `/NO<name>` overrides an earlier affirmative form and vice versa.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | The parameter or qualifier is present. |
| `STS$K_WARN` | It is absent. |

## __cli$cleanup

The Release Parse Context routine frees all memory owned by a parse context.

**Format** `int __cli$cleanup (clictx)`

**Returns** `int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| `clictx` | `CLI_CTX *` | modify | Address of the context to release. A `NULL` argument is accepted and ignored. |

Must be called once the parsed values are no longer needed. After the call the context pointer must not be used.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | Routine successfully completed. |

## __cli$show_verbs

The Display Command Definitions routine writes a human-readable listing of a command definition table to the log.

**Format** `void __cli$show_verbs (verbs, level)`

**Returns** None.

| Argument | Type | Access | Description |
|---|---|---|---|
| `verbs` | `CLI_VERB *` | read only | Address of the command table. |
| `level` | `int` | read only | Initial indentation level; pass 0 for the top-level table. |

Intended for documenting or debugging a command grammar; sub-verb chains are listed recursively with increasing indentation.

## __cli$help

The Help Generator routine prints a usage screen built from the command tables: the list of commands, or a verb's SYNOPSIS with its parameters and qualifiers.

**Format** `int __cli$help (CLI_VERB *verbs, CLI_VERB *verb)`

**Returns** `int` (write only) — STS$K_SUCCESS.

| Argument | Type | Access | Description |
|---|---|---|---|
| `verbs` | `CLI_VERB *` | read only | The top-level command table, used to list the commands when is NULL. |
| `verb` | `CLI_VERB *` | read only | The verb to describe, or NULL for an overview of all commands. |

Writes the usage text to the log destination (unprefixed). The parameter list shows each P*n* with its type; the qualifier list shows each name with its type, any default value and the `required`/`/NO`/`list` attributes.

The built-in help is requested by `/HELP`, `--help` or the short `-h`. The leading `/` or `-` is mandatory, so a bare `help` is an ordinary verb or parameter value and never a request for the screen. The short `-h` is kept as a deliberate alias, even though other Unix qualifiers use the `--` form. The request is detected by `__cli$parse()` before parameter validation; the relevant help is shown and the parse returns `STS$K_WARN`, so the caller skips `__cli$dispatch()`.

**Condition values**

`STS$K_SUCCESS` The help was printed.

# Chapter 4. Time and FAO Services (starlet) Reference

This chapter describes the `starlet` module - portable analogues of the OpenVMS time and formatting system services. The FAO engine writes into an `fao_desc_t` descriptor (a length/pointer pair; the length is the buffer capacity and is never exceeded). Each directive argument is an `fao_prm_t` (a machine word). Binary times are `fao_time_t` values: seconds of the UNIX epoch for an absolute time, a negative count for a delta time. The principal FAO directives are:

| | |
|---|---|
| `!AS !AZ !AC` | String by descriptor / zero-terminated / counted. |
| `!AD !AF` | Length and address; !AF shows non-printables as dots. |
| `!UB !UW !UL !UQ` | Unsigned decimal, 8/16/32/64-bit. |
| `!SB ... !SQ` | Signed decimal. |
| `!XB ... !XQ` | Hexadecimal. |
| `!OB ... !OQ` | Octal. |
| `!ZB ... !ZQ` | Zero-filled decimal. |
| `!%D !%T !%S` | Date / date-time / time from an fao_time_t pointer. |
| `!n*c` | Repeat character c, n times. |
| `!n(...!)` | Repeat a directive group n times. |
| `!n<...!>` | Fixed-width field of n columns. |
| `!/ !_ !^ !!` | Newline, tab, form feed, literal "!". |

The `!xQ` quadword and `!%D/!%T` time directives are passed *by value*: `fao_prm_t` is a 64-bit type, so a quadword or a time fits directly in a parameter on every target. All routines of this module carry the `__util$` prefix. The FAO routines (`__util$fao`/`__util$faol`) replaced the old FAO from `utility_routines`.

## __util$fao

The Formatted ASCII Output routine formats a variable argument list into a caller buffer under control of a VMS-style FAO control string.

**Format** `int __util$fao (ctlstr, outlen, outbuf [, prm, ...])`

**Returns** `int` (write only) — A status code: `STS$K_SUCCESS`, or `FAO$K_BUFOVF` if the result was truncated.

| Argument | Type | Access | Description |
|---|---|---|---|
| `ctlstr` | `const char *` | read only | Zero-terminated FAO control string. Directives are introduced by `!`; see the table at the head of this chapter. |
| `outlen` | `unsigned short *` | write only | Receives the number of bytes written to the buffer. May be `NULL`. |
| `outbuf` | `fao_desc_t *` | modify | Descriptor of the output buffer. `dsc$w_length` gives the capacity on input; the buffer is never written beyond it. |
| `prm` | `fao_prm_t` | read only | Directive arguments. Each must be cast to `fao_prm_t`. Quadword (`!xQ`) and time (`!%T`/`!%D`) directives are passed BY VALUE (`fao_prm_t` is 64 bits wide). |

The portable analogue of OpenVMS `SYS$FAO`. Output is bounded by the descriptor capacity; on overflow the result is truncated and `FAO$K_BUFOVF` is returned. Up to `FAO$K_MAXPRM` arguments are cached so that parameter-reuse directives (`!n-`) work.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | The string was formatted. |
| `FAO$K_BUFOVF` | The output buffer was too small; the result is truncated. |
| `FAO$K_BADDIR` | A malformed directive was encountered. |
| `STS$K_ERROR` | A required argument was `NULL`. |

## __util$faol

The Formatted ASCII Output with List routine is identical to __util$fao except that the directive arguments are supplied as an explicit array.

**Format** `int __util$faol (ctlstr, outlen, outbuf, prmlst, prmcnt)`

**Returns** `int` (write only) — A status code, as for `__util$fao`.

| Argument | Type | Access | Description |
|---|---|---|---|
| `ctlstr` | `const char *` | read only | FAO control string. |
| `outlen` | `unsigned short *` | write only | Receives the number of bytes written; may be `NULL`. |
| `outbuf` | `fao_desc_t *` | modify | Output buffer descriptor. |
| `prmlst` | `const fao_prm_t *` | read only | Array of directive arguments. |
| `prmcnt` | `int` | read only | Number of elements in **prmlst**. Requests beyond the array yield zero. |

The portable analogue of OpenVMS `SYS$FAOL`. Preferred when the argument count is determined at run time.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | The string was formatted. |
| `FAO$K_BUFOVF` | The output was truncated. |
| `FAO$K_BADDIR` | A malformed directive was encountered. |

## __util$bintim

The Convert ASCII Time to Binary routine parses an OpenVMS-style absolute or delta time string into a binary time value.

**Format** `int __util$bintim (timstr, timadr)`

**Returns** `int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| `timstr` | `const char *` | read only | Zero-terminated time string. Absolute: `[dd]-MMM-[yyyy] [hh:mm:ss[.cc]]` or a time-of-day alone. Delta: `dddd hh:mm:ss`, a day count and a time of day. |
| `timadr` | `fao_time_t *` | write only | Receives the binary time: seconds of the UNIX epoch for an absolute time, a negative seconds count for a delta time. |

The portable analogue of OpenVMS `SYS$BINTIM`. Omitted absolute fields default to the current date. The day is validated against the month length with leap-year handling. By convention delta times are represented as negative values, which is what `__util$add_times` and `__util$sub_times` expect.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | The string was parsed. |
| `FAO$K_IVTIME` | The string is not a valid time. |
| `STS$K_ERROR` | A `NULL` argument was supplied. |

## __util$gettim

The Get System Time routine returns the current time as a binary time value.

**Format** `int __util$gettim (timadr)`

**Returns** `int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| `timadr` | `fao_time_t *` | write only | Receives the current absolute time (seconds of the UNIX epoch). |

The portable analogue of OpenVMS `SYS$GETTIM`. The underlying clock source is the hosted C run-time by default and may be overridden at build time through the `FAO$GETTIME` macro.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | Routine successfully completed. |
| `STS$K_ERROR` | A `NULL` argument was supplied. |

## __util$asctim

The Convert Binary Time to ASCII routine formats a binary time value into a readable string.

**Format** `int __util$asctim (outlen, outbuf, timadr, cvtflg)`

**Returns** `int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| `outlen` | `unsigned short *` | write only | Receives the number of bytes written; may be `NULL`. |
| `outbuf` | `fao_desc_t *` | modify | Output buffer descriptor. |
| `timadr` | `const fao_time_t *` | read only | Address of the binary time to format. |
| `cvtflg` | `int` | read only | Conversion flag: 0 formats the full `dd-MMM-yyyy hh:mm:ss.cc`, non-zero formats the time of day only. |

The portable analogue of OpenVMS `SYS$ASCTIM`. Output is bounded by the descriptor capacity; a value produced by `__util$bintim` is reproduced exactly.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | The time was formatted. |
| `FAO$K_BUFOVF` | The output buffer was too small; the result is truncated. |
| `STS$K_ERROR` | A `NULL` argument was supplied. |

## __util$add_times

The Add Two Binary Times routine adds two binary times following the OpenVMS combination rules for absolute and delta times.

**Format** `int __util$add_times (tim1, tim2, result)`

**Returns** `int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| `tim1` | `const fao_time_t *` | read only | First operand. |
| `tim2` | `const fao_time_t *` | read only | Second operand. |
| `result` | `fao_time_t *` | write only | Receives the sum. |

The portable analogue of OpenVMS `LIB$ADD_TIMES`. Delta times are negative, absolute times non-negative. Two deltas add to a delta; an absolute and a delta add to an absolute advanced by the delta magnitude; two absolutes are rejected.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | The addition completed. |
| `FAO$K_IVTIME` | Both operands are absolute times. |
| `STS$K_ERROR` | A `NULL` argument was supplied. |

## __util$sub_times

The Subtract Two Binary Times routine subtracts the second binary time from the first following the OpenVMS combination rules.

**Format** `int __util$sub_times (tim1, tim2, result)`

**Returns** `int` (write only) — A status code.

| Argument | Type | Access | Description |
|---|---|---|---|
| `tim1` | `const fao_time_t *` | read only | Minuend. |
| `tim2` | `const fao_time_t *` | read only | Subtrahend. |
| `result` | `fao_time_t *` | write only | Receives the difference. |

The portable analogue of OpenVMS `LIB$SUB_TIMES`. Absolute minus absolute yields a delta (and **tim1** must not precede **tim2**); absolute minus delta yields an absolute; delta minus delta yields a delta; delta minus absolute is rejected.

**Condition values**

| | |
|---|---|
| `STS$K_SUCCESS` | The subtraction completed. |
| `FAO$K_NEGTIM` | The result would be negative. |
| `FAO$K_IVTIME` | A delta-minus-absolute subtraction was requested. |
| `STS$K_ERROR` | A `NULL` argument was supplied. |

# Chapter 5. Programming Examples

The following complete programs illustrate how the routines are combined in practice. Each was compiled against the modules and executed; the output shown is the actual result, with the timestamp and process-id prefix of each log record omitted for brevity.

## Example A.1. First application: options, configuration and logging

Program skeleton: declare an option table, parse the command line and a configuration file with the same table, and write to the log.

```
/*
**  Example A.1 — First application: options, configuration and logging
**
**  File:  app.c
**  Build:  cc -std=gnu11 -O2 -I. app.c utility_routines.c -o app
**  Run:  ./app
*/
```

```
#define __MODULE__ "APP$"          /* the module name is printed in every log line */
#include "utility_routines.h"

int main(int argc, char **argv)
{
int             l_trace = 0;
unsigned short  l_port  = 0;
ASC             l_logfile = {0}, l_config = {0};

    /* Option table: name, value sink, sink size, type. */
    /* OPTS_NULL terminates the table.                    */
    const OPTS l_opts[] = {
        { $ASCINI("config"),  &l_config,  ASC$K_SZ, OPTS$K_CONF },
        { $ASCINI("port"),    &l_port,    0,        OPTS$K_PORT },
        { $ASCINI("logfile"), &l_logfile, ASC$K_SZ, OPTS$K_STR  },
        { $ASCINI("trace"),   &l_trace,   0,        OPTS$K_OPT  },
        OPTS_NULL
    };

    /* Parse argv. A -config= option immediately reads the config file with the same
table. */
    /* Test (1 & status): the low status bit is set on success.                */
    if ( !(1 & __util$getparams(argc, argv, (OPTS *) l_opts)) )
        return  $LOG(STS$K_FATAL, "invalid command line");

    /* If a log file was given, redirect the log there */
    if ( $ASCLEN(&l_logfile) )
        {
        char l_path[256];
        __util$asc2str(&l_logfile, l_path);
        __util$deflog(l_path, NULL);
        }

    __util$showparams(l_opts);
    $LOG(STS$K_INFO, "app started on port %d", l_port);   /* STS$K_INFO -> letter I */
    return  STS$K_SUCCESS;
}
```

### Output

```
%UTILS-I:  trace = OFF
%APP-I:  app started on port 0, trace=0
( a real invocation: app -config=app.conf -port=8080 -trace )
```

## Example A.2. Working with time (starlet): parse, arithmetic, format

Turn a time string into a number, add an interval, and format it back. Rule: an absolute time (a moment) is non-negative, while a delta (interval) is stored as a negative number.

```
/*
**  Example A.2 — Working with time (starlet): parse, arithmetic, format
**
**  File:  tim.c
**  Build:  cc -std=gnu11 -O2 -I. tim.c starlet.c -o tim
**  Run:  ./tim
*/
```

```
#include <stdio.h>
#include "utility_routines.h"
#include "starlet.h"

int main(void)
{
fao_time_t     l_t, l_delta, l_later;
char           l_buf [40];
fao_desc_t     l_dsc = $FAO_DESC(l_buf);      /* the descriptor holds the buffer size
*/
unsigned short l_len;

    /* Absolute time (a moment) -> epoch seconds; yields 1539616633 */
    __util$bintim("15-OCT-2018 15:17:13", &l_t);

    /* Interval "1 day 12 hours". Format "days h:m:s".              */
    /* a delta is stored NEGATIVE: yields -129600.                 */
    __util$bintim("1 12:00:00", &l_delta);

    /* moment + delta = a new moment, advanced by 1.5 days */
    __util$add_times(&l_t, &l_delta, &l_later);

    /* Number -> string. Flag 0 = "date and time". The buffer cannot overflow - */
    /* its size was recorded in the descriptor by the $FAO_DESC macro.        */
    __util$asctim(&l_len, &l_dsc, &l_later, 0);
    l_buf[l_len] = '\0';

    printf("t+delta = %s\n", l_buf);
    return 0;
}
```

### Output

```
t+delta = 17-OCT-2018 03:17:13.00

( 15-OCT 15:17 + 1.5 days = 17-OCT 03:17 )
```

## Example A.3. Integrity check: CRC-32 and hexadecimal

Compute a checksum over a buffer and print it in hexadecimal.

```
/*
**  Example A.3 — Integrity check: CRC-32 and hexadecimal
**
**  File:  crc.c
**  Build:  cc -std=gnu11 -O2 -I. crc.c utility_routines.c -o crc
**  Run:  ./crc
*/
```

```
#define __MODULE__ "CRC$"
#include "utility_routines.h"

int main(void)
{
unsigned char  l_frame[] = { 0x02, 0x10, 0x00, 0x04, 0xDE, 0xAD, 0xBE, 0xEF };
unsigned       l_crc;
char           l_hex[2 * sizeof(l_frame) + 1];

    /* CRC-32 over the whole array. 0 is the seed (a fresh checksum).       */
    /* For a stream: pass the result as the seed of the next call           */
    l_crc = __util$crc32c(0, l_frame, sizeof(l_frame));

    /* Bytes -> hex string (a byte = two chars). Sink: 2*N + 1 bytes.  */
    __util$bin2hex(l_frame, l_hex, sizeof(l_frame));

    /* The crc32c name is historical: it computes the standard CRC-32 (as in zlib) */
    $LOG(STS$K_INFO, "frame [%s] crc32=0x%08x", l_hex, l_crc);
    return  STS$K_SUCCESS;
}
```

### Output

```
frame [02100004deadbeef] crc32=0xe8819e5c
```

## Example A.4. A work queue

A minimal producer/consumer pipeline on a doubly-linked queue.

```
/*
**  Example A.4 — A work queue
**
**  File:  que.c
**  Build:  cc -std=gnu11 -O2 -I. que.c utility_routines.c -o que
**  Run:  ./que
*/
```

```
#define __MODULE__ "QUE$"
#include "utility_routines.h"

/* A queue element MUST start with an ENTRY - the pair of links */
/* the queue uses to chain elements.                            */
typedef struct { ENTRY links; int value; } WORK_ITEM;

int main(void)
{
__QUEUE    l_q = QUEUE_INITIALIZER;
WORK_ITEM  l_pool[8] = {0};        /* = {0} is required: insertion checks a
housekeeping field */
unsigned   l_count;
WORK_ITEM *l_item;

    /* Insert 8 elements at the tail. l_count returns the count before insertion. */
    for ( int i = 0; i < 8; i++ )
        {
        l_pool[i].value = i * 100;
        $INSQTAIL(&l_q, &l_pool[i], &l_count);
        }

    /* Remove from the head (FIFO). $REMQHEAD returns "success" even on an empty */
    /* queue, so the && l_count guard is REQUIRED - otherwise the loop never ends. */
    while ( (1 & $REMQHEAD(&l_q, &l_item, &l_count)) && l_count )
        $LOG(STS$K_INFO, "value=%d", l_item->value);
    return  STS$K_SUCCESS;
}
```

### Output

```
%UTILS-I:  processing item value=0
%UTILS-I:  processing item value=100
%UTILS-I:  processing item value=200
...  (through value=700)
```

## Example A.5. Command-line parsing (CLI): verb, parameters, qualifiers

Describe a command like `COPY source dest /verbose /port=8080` and parse it. A command is a verb plus positional parameters P1..P8 plus `/name[=value]` qualifiers.

```
/*
**  Example A.5 — Command-line parsing (CLI): verb, parameters, qualifiers
**
**  File:  cli.c
**  Build:  cc -std=gnu11 -O2 -I. cli.c cli_routines.c utility_routines.c -o cli
**  Run:  ./cli
*/
```

```
#define __MODULE__ "DEMO$"
#include "utility_routines.h"
#include "cli_routines.h"

static CLI_PQDESC copy_params[];

/* Action routine invoked for the COPY verb */
static int copy_action(CLI_CTX *a_ctx, void *a_arg)
{
ASC l_src = {0};

    (void) a_arg;
    /* Fetch the P1 (source) value from the parsed context */
    if ( 1 & __cli$get_value(a_ctx, &copy_params[0], &l_src) )
        $LOG(STS$K_INFO, "COPY source = %.*s", $ASC(&l_src));
    return STS$K_SUCCESS;
}

/* P1 = source, P2 = dest (both are file names) */
/* Positional parameters: source = P1, dest = P2; type FILE = a file name */
static CLI_PQDESC copy_params[] = {
    { .name = {$ASCINI("source")}, .type = CLI$K_FILE, .pn = CLI$K_P1 },
    { .name = {$ASCINI("dest")},   .type = CLI$K_FILE, .pn = CLI$K_P2 },
    { {0} }
};
/* /VERBOSE (flag), /PORT=<number> */
/* Qualifiers: /verbose is a flag (OPT), /port is a number (NUM, validated by type) */
static CLI_PQDESC copy_quals[] = {
    { .name = {$ASCINI("verbose")}, .type = CLI$K_OPT },
    { .name = {$ASCINI("port")},    .type = CLI$K_NUM },
    { {0} }
};
/* Verb table: binds "copy" to its parameters, qualifiers and action */
static CLI_VERB commands[] = {
    { .name = {$ASCINI("copy")}, .params = copy_params, .quals = copy_quals, .act_rtn =
copy_action },
    { {0} }
};

int main(int argc, char **argv)
{
void *l_ctx = NULL;

    /* Parse argv. CLI$M_OPSIGNAL prints a message on each error.            */
    if ( !(1 & __cli$parse(commands, CLI$M_OPSIGNAL, argc-1, argv+1, &l_ctx)) )
        return  STS$K_ERROR;

    __cli$dispatch(l_ctx);   /* call copy_action; it uses __cli$get_value inside */
    __cli$cleanup(l_ctx);
```

```
    return  STS$K_SUCCESS;
}
```

### Output

```
$ COPY a.txt b.txt /verbose /port=8080
%UTILS-I:  COPY source = a.txt

$ COPY a.txt b.txt /port=abc
%CLI_RTNS-E:  Value 'abc' is not a valid number   (rejected)
```

## Example A.6. Binary protocol avproto: build and parse a packet

Build a packet from Tag-Length-Value records and parse it on the other side. All numbers are held in network byte order.

```
/*
**  Example A.6 — Binary protocol avproto: build and parse a packet
**
**  File:  avp.c
**  Build:  cc -std=gnu11 -O2 -I. avp.c avproto.c utility_routines.c -o avp
**  Run:  ./avp
*/
```

```
#define __MODULE__ "AVP$"
#define __FAC__    "AVP"
#include <stdint.h>
#include <arpa/inet.h>
#include "utility_routines.h"
#include "avproto.h"

#define APP$K_TAG_ID    10
#define APP$K_TAG_NAME  11

/* Build a request: return the length of the ready PDU */
static unsigned build_request (void *a_pdu, unsigned a_pdusz)
{
AVPROTO_PDU *l_pdu = (AVPROTO_PDU *) a_pdu;
uint32_t     l_id  = 0x1234ABCD;

    /* Header: 8-byte signature (exactly 8, no embedded NUL!), csr, seq. */
    /* hset clears the length, so it is called BEFORE the first __tlv$put.   */
    __tlv$hset(l_pdu, (const unsigned char *) "TLV-PDU1", 1, 42);
    /* A TLV record: tag, type, value. Here a 32-bit number, then a byte block */
    __tlv$put(l_pdu, a_pdusz, APP$K_TAG_ID, TAG$K_LONGWORD, &l_id, 4);
    __tlv$put(l_pdu, a_pdusz, APP$K_TAG_NAME, TAG$K_BBLOCK,   "demo-c", 6);

    return  sizeof(AVPROTO_HDR) + ntohl(l_pdu->r_hdr.u_len);
}

/* Parse a received PDU */
static int parse_request (void *a_pdu, unsigned a_rcvlen)
{
unsigned  l_ulen, l_ucsr, l_useq, l_vtype, l_vsz;
uint32_t  l_id;
char      l_name[64];
int       l_ctx;

    /* Validate the signature and extract the header fields */
    if ( !(1 & __tlv$hget(a_pdu, (const unsigned char *) "TLV-PDU1", &l_ulen, &l_ucsr,
&l_useq)) )
        return  $LOG(STS$K_ERROR, "foreign or corrupted PDU");

    /* Longword by tag APP$K_TAG_ID */
    l_ctx = -1; l_vsz = sizeof(l_id);
    /* a_rcvlen is the REAL received size: every record is bounds-checked against it,
*/
    /* so that a malformed packet cannot run past the buffer.                       */
    if ( 1 & __tlv$get(a_pdu, a_rcvlen, &l_ctx, APP$K_TAG_ID, &l_vtype, &l_id,
&l_vsz) )
        $LOG(STS$K_INFO, "id = 0x%08x", l_id);

    /* Block by tag APP$K_TAG_NAME (valsz is mandatory for variable length) */
    l_ctx = -1; l_vsz = sizeof(l_name);
    if ( 1 & __tlv$get(a_pdu, a_rcvlen, &l_ctx, APP$K_TAG_NAME, &l_vtype, l_name,
```

```
&l_vsz) )
        $LOG(STS$K_INFO, "name = %.*s", l_vsz, l_name);

    return  STS$K_SUCCESS;
}

int main(void)
{
char     l_buf[8192];
unsigned l_len;

    l_len = build_request(l_buf, sizeof(l_buf));     /* sender */
    $LOG(STS$K_INFO, "built PDU of %u octets", l_len);

    parse_request(l_buf, l_len);                     /* receiver (l_len is the real
size) */
    __tlv$dump(l_buf, l_len);
    return  STS$K_SUCCESS;
}
```

### Output

```
%AVP-I:  built PDU of 38 octets
%AVP-I:  id = 0x1234abcd
%AVP-I:  name = demo-c
%UTILS-I:  PDU [len=18, csr=0x1, seq=42]
%UTILS-I:  [0000] TLV [tag=400a(id=000a, type=0004), len=04] 0x400a0004:0x1234abcd
%UTILS-I:  [0001] TLV [tag=000b(id=000b, type=0000), len=06] 0x000b0006:0x64656d6f2d63
```

# Appendix A. FAO Directives Reference

The FAO service of the `starlet` module renders a control string with embedded directives. Every parameter travels as a 64-bit `fao_prm_t`; a pointer is passed as `(fao_prm_t) (uintptr_t) ptr`. The `$PUTMSG_FAO()` and `$LOG_FAO()` macros apply the cast and derive the parameter count for the caller.

## A.1 Directive summary

| Directive | Description |
|---|---|
| `!AS` | String by descriptor (fao_desc_t *). |
| `!AZ` | Zero-terminated string (char *). |
| `!AC` | Counted string, the first byte is the length. |
| `!AD` | Two parameters: length, address. |
| `!AF` | As !AD, non-printables are shown as '.'. |
| `!UB !UW !UL !UQ` | Unsigned decimal, 8/16/32/64 bit. |
| `!SB !SW !SL !SQ` | Signed decimal. |
| `!XB !XW !XL !XQ` | Hexadecimal, zero-filled to 2/4/8/16 digits. |
| `!OB !OW !OL !OQ` | Octal, zero-filled. |
| `!ZB !ZW !ZL !ZQ` | Unsigned decimal, zero-filled. |
| `!/ !_ !^ !!` | CR+LF; horizontal tab; form feed; literal '!'. |
| `!%S` | 's' when the last converted number is not 1. |
| `!%T` | Time, hh:mm:ss.cc (UTC + FAO$K_TZOFFSET). |
| `!%D` | Date, dd-MMM-yyyy hh:mm:ss.cc. |
| `!%M` | MAC / hardware address, aa:bb:cc:dd:ee:ff — a pointer to 6 octets. |
| `!%I` | IPv4 address, d.d.d.d — a pointer to 4 octets. |
| `!%6` | IPv6, RFC 5952 canonical; an IPv4-mapped one in the mixed ::ffff:a.b.c.d notation — a pointer to 16 octets. |
| `!%U` | UUID, 8-4-4-4-12 — a pointer to 16 octets. |
| `!nXL ...` | Explicit output field width for a directive. |
| `!n*c` | Repeat the character c, n times. |
| `!n(...)` | Repeat the directive group n times. |
| `!n<...!>` | Fixed-width field: blank-filled or truncated to n columns. |
| `!n-` | Reuse the last n parameters. |
| `!n+` | Skip the next n parameters. |

**NOTE**
The address directives take a POINTER to the raw octets (network order); a NULL pointer yields `(null)`. The IPv6 form follows RFC 5952: the longest run of two or more zero groups is compressed to `::` (the leftmost on a tie), a lone zero group stays.

# Appendix B. Additions since the reference corpus

## B.1 $LOG_FAO - logging under an FAO control string

```
$LOG_FAO(severity, ctl, ...)
```

The FAO twin of `$LOG`: the body is an FAO control string rendered by `__util$faol()`, so every directive of Appendix A - including the address ones - is at hand. The macro casts each parameter to `fao_prm_t` and derives the count itself, exactly the `$PUTMSG_FAO` ergonomics; the zero-parameter form is legal. The record carries the usual timestamp / `%<facility>-<S>:` envelope (and the `[module\function:line]` descriptor in a debug build); `STS$M_SYSLOG` forwards the already-formatted body to syslog. The routines behind the macro are `__util$log_fao()` and `__util$logd_fao()`.

```
$LOG_FAO(STS$K_INFO, "if=!AZ rx=!UL peer=!%I", "eth0", 777, l_ip4);
  /* 29-08-2026 17:12:10.013   765 [MOD\main:47] %TFAC-I:  if=eth0 rx=777
peer=192.168.1.10 */
```

## B.2 Wildcard matching - __util$pattern_match()

The matcher is iterative with a single backtrack point, bounded by O(|string| * |pattern|) - hostile multi-star patterns cannot blow it up. Wildcards:

| Wildcard | Meaning |
|---|---|
| * | Any run of octets, including the empty one. |
| % | Exactly one octet. |
| ? | An alias of %, exactly one octet (the DOS/glob spelling). A literal question mark in a pattern is therefore a wildcard. |

## B.3 The facility name lives in the record

`EMSG_RECORD` carries a trailing `facname` field, stamped into every record by `__util$inimsg()` at registration; `__util$putmsg_fao()`/`__util$putmsgd_fao()` read the name off the found record - no second walk over the descriptor list per message. A NULL or empty name shows as `NONAME`. Existing positional record initializers stay valid: the field is trailing and is filled by the library, not by the caller.

## B.4 The log-record prefix

The "DD-MM-YYYY HH:MM:SS.msc <tid> " prefix is rendered by a cached helper: the date-time part is reformatted only when the second changes, and the thread id is fetched from the kernel once per thread. End-to-end logging is about four times faster for it; the bytes are identical to the historical rendering.

> **NOTE**
> After a fork() the child inherits the parent's cached thread id; a process that forks after its first log record and cares about the tid column should reset it via pthread_atfork() or re-exec.